AI Engineering Boot Camp

Day 9 — Matplotlib & Data Visualisation

Line plots · Scatter · Histograms · Heatmaps · Subplots

Target: Google · Meta · OpenAI · Anthropic

Files You Will Create Today

- day9_line_plots.py — training loss curves, accuracy over epochs
- day9_scatter_hist.py — scatter plots, histograms, distributions
- day9_bar_heatmap.py — bar charts, correlation heatmap
- day9_subplots.py — combining multiple charts in one figure

> *All files save charts as .png files to your PythonWork folder. All pass mypy --strict. All pushed to GitHub.*

Why Matplotlib?

You can't understand 10,000 rows of numbers. But a histogram shows the distribution instantly. A loss curve tells you if your model is overfitting in seconds. A correlation heatmap shows which features matter at a glance.

At Google and Anthropic, every experiment produces charts. Results without visualisation are harder to trust and harder to communicate to the rest of the team.

Chart type	Used for in ML
Line plot	Training loss and accuracy over epochs — see if model is learning
Scatter plot	Relationship between two features — spot clusters and outliers
Histogram	Distribution of a feature — check for skew, outliers, normal dist
Bar chart	Comparing categories — feature importance, class counts
Heatmap	Correlation matrix — which features predict the target
Subplots	Multiple charts side by side — compare train vs validation

Figure and Axes — The Core Concept

Everything in Matplotlib has two parts:

- Figure — the whole canvas. Like a blank sheet of paper.
- Axes — one individual plot on that canvas. A Figure can have many Axes.

```
import matplotlib.pyplot as plt

# One plot
fig, ax = plt.subplots()

# Grid of 6 plots — 2 rows, 3 columns
fig, axes = plt.subplots(2, 3, figsize=(12, 8))
```

> *Always use fig, ax = plt.subplots() — not plt.plot() directly. The object-oriented approach gives you full control and is what production code uses.*

File 1 — day9_line_plots.py

What it does

Simulates a training run and plots loss and accuracy over 50 epochs. This is the most common chart in all of ML — you look at it after every training run.

Key code patterns

```
import matplotlib.pyplot as plt
import numpy as np

# Simulate training data
epochs = np.arange(1, 51)
train_loss = 2.0 * np.exp(-0.05 * epochs) + np.random.normal(0, 0.05, 50)
val_loss   = 2.0 * np.exp(-0.04 * epochs) + np.random.normal(0, 0.08, 50)

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(epochs, train_loss, label='train loss', color='blue')
ax.plot(epochs, val_loss,   label='val loss',   color='orange', linestyle='--')
ax.set_xlabel('Epoch')
ax.set_ylabel('Loss')
ax.set_title('Training vs Validation Loss')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('loss_curve.png', dpi=150)
plt.close()
```

Key concepts

- figsize=(10, 5) — width and height in inches
- label= — text shown in the legend
- linestyle='--' — dashed line for validation
- ax.grid(True, alpha=0.3) — subtle grid, alpha controls transparency
- plt.savefig() — save to file instead of displaying
- plt.close() — free memory after saving

What to look for in a loss curve

Pattern	What it means
Both curves decrease together	Model is learning correctly
Train loss down, val loss up	Overfitting — model memorising training data
Both curves flat from epoch 1	Learning rate too low — model not learning
Loss jumps up and down	Learning rate too high — unstable training

File 2 — day9_scatter_hist.py

What it does

Creates scatter plots to show relationships between features and histograms to show distributions. You run these on every new dataset before training anything.

Scatter plot

```
fig, ax = plt.subplots(figsize=(8, 6))
scatter = ax.scatter(
    df['experience_years'],
    df['income'],
    c=df['hired'].astype(int),  # colour by class
    cmap='coolwarm',            # blue=0, red=1
    alpha=0.6,                  # transparency
    edgecolors='white',
    linewidth=0.5
)
```

```
plt.colorbar(scatter, label='Hired')
```

Histogram

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.hist(df['income'], bins=30, color='steelblue',
        edgecolor='white', alpha=0.8)
ax.axvline(df['income'].mean(), color='red',
           linestyle='--', label='mean')
ax.axvline(df['income'].median(), color='green',
           linestyle='--', label='median')
```

Key concepts

- c=df['hired'].astype(int) — colour each point by class label
- cmap='coolwarm' — colour map: blue for 0, red for 1
- alpha=0.6 — transparency, helps see overlapping points
- bins=30 — number of bars in histogram
- ax.axvline() — vertical line at a specific x value

File 3 — day9_bar_heatmap.py

What it does

Bar chart for comparing category counts or feature importance. Heatmap for visualising the correlation matrix — the most useful chart for understanding which features matter.

Bar chart

```
categories = ['A', 'B', 'C', 'D']
values = [23, 45, 12, 67]
```

```
fig, ax = plt.subplots(figsize=(8, 5))
bars = ax.bar(categories, values,
              color='steelblue', edgecolor='white',
              width=0.6)
# Add value labels on top of each bar
for bar, val in zip(bars, values):
```

```
    ax.text(bar.get_x() + bar.get_width()/2,
            bar.get_height() + 1,
            str(val), ha='center', fontsize=10)
```

Heatmap — correlation matrix

```
corr = df.corr(numeric_only=True)

fig, ax = plt.subplots(figsize=(8, 6))
im = ax.imshow(corr.values, cmap='RdBu_r',
               vmin=-1, vmax=1, aspect='auto')
plt.colorbar(im, ax=ax)
ax.set_xticks(range(len(corr.columns)))
ax.set_yticks(range(len(corr.columns)))
ax.set_xticklabels(corr.columns, rotation=45, ha='right')
ax.set_yticklabels(corr.columns)
# Add correlation values as text
for i in range(len(corr)):
    for j in range(len(corr.columns)):
        ax.text(j, i, f'{corr.values[i,j]:.2f}',
                ha='center', va='center', fontsize=8)
```

Key concepts

- ax.imshow() — displays a 2D array as a colour image
- cmap='RdBu_r' — red for positive, blue for negative correlation
- vmin=-1, vmax=1 — fixes colour scale to correlation range
- rotation=45 — rotates x-axis labels so they don't overlap

File 4 — day9_subplots.py

What it does

Combines multiple charts in one figure. This is the standard format for ML experiment reports — loss curve, accuracy curve, and data distributions all in one image.

Creating a 2x2 grid of plots

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle('Training Report', fontsize=16, fontweight='bold')

# Top left — loss curve
axes[0, 0].plot(epochs, train_loss, label='train')
axes[0, 0].plot(epochs, val_loss, label='val')
axes[0, 0].set_title('Loss')
axes[0, 0].legend()

# Top right — accuracy curve
axes[0, 1].plot(epochs, train_acc, label='train', color='green')
axes[0, 1].plot(epochs, val_acc, label='val', color='lime')
axes[0, 1].set_title('Accuracy')
axes[0, 1].legend()

# Bottom left — histogram
axes[1, 0].hist(data, bins=30, color='steelblue')
axes[1, 0].set_title('Distribution')

# Bottom right — scatter
axes[1, 1].scatter(x, y, alpha=0.5, color='purple')
axes[1, 1].set_title('Feature Relationship')

plt.tight_layout()
plt.savefig('training_report.png', dpi=150, bbox_inches='tight')
plt.close()
```

Key concepts

- axes[row, col] — access each subplot by row and column index
- fig.suptitle() — title for the entire figure, not just one subplot
- plt.tight_layout() — automatically adjusts spacing so plots don't overlap
- bbox_inches='tight' — prevents labels being cut off when saving

Quick Reference — Matplotlib Cheatsheet

Essential methods

Method	What it does
ax.plot(x, y)	Line plot
ax.scatter(x, y)	Scatter plot
ax.hist(x, bins=30)	Histogram
ax.bar(categories, values)	Bar chart
ax.imshow(matrix, cmap=...)	Heatmap / image
ax.set_title('text')	Set plot title
ax.set_xlabel('text')	Set x-axis label
ax.set_ylabel('text')	Set y-axis label
ax.legend()	Show legend (uses label= from plots)
ax.grid(True, alpha=0.3)	Add subtle grid
ax.axvline(x, color=...)	Vertical line at x
ax.axhline(y, color=...)	Horizontal line at y
plt.tight_layout()	Fix spacing between subplots
plt.savefig('file.png', dpi=150)	Save to file
plt.close()	Free memory after saving

Common colour maps (cmap)

cmap	Use for
'coolwarm'	Diverging — good for scatter coloured by class
'RdBu_r'	Red-Blue — standard for correlation heatmaps
'viridis'	Sequential — general purpose, colourblind friendly
'Blues'	Single colour sequential — good for counts

Anki Cards — Day 9

- Q: What is a Figure in Matplotlib? A: The whole canvas — contains one or more Axes
- Q: What is an Axes in Matplotlib? A: One individual plot on the Figure
- Q: How do you create a 2x2 grid of plots? A: fig, axes = plt.subplots(2, 2, figsize=(12, 10))
- Q: What does plt.tight_layout() do? A: Automatically adjusts spacing so plots don't overlap
- Q: What does ax.axvline() do? A: Draws a vertical line at a specific x value
- Q: What chart shows training overfitting? A: Loss curve — train loss down, val loss up
- Q: What does alpha= control? A: Transparency — 0 is invisible, 1 is solid
- Q: Why use plt.close() after saving? A: Frees memory — prevents figures accumulating
- Q: What cmap is standard for correlation heatmaps? A: RdBu_r — red positive, blue negative
- Q: What does bbox_inches='tight' do in savefig? A: Prevents labels being cut off at the edges

---

After Day 9: Day 10 — scikit-learn and your first real ML model trained on real data.